

DDA3020 Machine Learning: Lecture 17 Principal Component Analysis

Baoyuan Wu
School of Artificial Intelligence, CUHK-SZ

April 27/29, 2026

Outline

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Preliminary: Projection onto a subspace

- Given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$ with $\mathbf{x}^{(n)} \in \mathbb{R}^D$ and D being the original dimension. And we define the mean as $\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} \in \mathbb{R}^D$.
- K -dimensional subspace $\mathcal{S}$ is spanned by an orthonormal basis $\{\mathbf{u}_k\}_{k=1}^K$ with $\mathbf{u}_k \in \mathbb{R}^D$.
- $\|\mathbf{u}_k\| = 1, \forall k; \mathbf{u}_i^\top \mathbf{u}_j = 0$ if $i \neq j, \forall i, j$.
- Approximate each data point $\mathbf{x} \in \mathbb{R}^D$ as:

$$\tilde{\mathbf{x}} = \boldsymbol{\mu} + \text{Proj}_{\mathcal{S}}(\mathbf{x} - \boldsymbol{\mu}) = \boldsymbol{\mu} + \sum_{k=1}^K z_k \mathbf{u}_k,$$

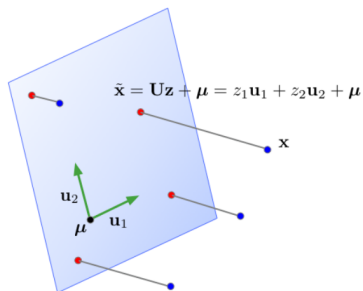
where $z_k = \mathbf{u}_k^\top (\mathbf{x} - \boldsymbol{\mu})$ can be seen as the projection length of $\mathbf{x} - \boldsymbol{\mu}$ on the k -th basis $\mathbf{u}_k$.

- Let $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_K] \in \mathbb{R}^{D \times K}$ be a matrix with columns $\{\mathbf{u}_k\}_{k=1}^K$, then

$$\tilde{\mathbf{x}} = \boldsymbol{\mu} + \mathbf{U}\mathbf{z} \in \mathbb{R}^D, \text{ which is called reconstruction of } \mathbf{x} \quad (1)$$

$$\mathbf{z} = \mathbf{U}^\top (\mathbf{x} - \boldsymbol{\mu}) \in \mathbb{R}^K, \text{ which is called representation/code of } \mathbf{x}. \quad (2)$$

Preliminary: Projection onto a subspace



$$\mathbf{z} = \mathbf{U}^\top(\mathbf{x} - \boldsymbol{\mu})$$

- In the above example, the blue point $\mathbf{x} \in \mathbb{R}^3$ is projected onto a 2-dimensional subspace $\mathcal{S}$ spanned by 2 basis vectors $\{\mathbf{u}_1, \mathbf{u}_2\}$. And, the mean vector of all blue points $\boldsymbol{\mu} \in \mathbb{R}^3$ is set as the origin of $\mathcal{S}$.
- Through projection, each blue point $\mathbf{x}$ has a reconstruction $\tilde{\mathbf{x}} \in \mathbb{R}^3$, which locates in $\mathcal{S}$.
- The coordinate value of $\tilde{\mathbf{x}}$ in the new coordinate system $\{\mathbf{u}_1, \mathbf{u}_2\}$ is represented by $\mathbf{z} \in \mathbb{R}^2$.

Preliminary: Projection onto a subspace

Theorem (Orthogonal theorem)

The vector $\mathbf{x} - \tilde{\mathbf{x}}$ is *orthogonal* to the subspace $\mathcal{S}$, i.e.,

$$\mathbf{U}^\top (\mathbf{x} - \tilde{\mathbf{x}}) = \mathbf{0}.$$

Proof:

- Utilizing the definition of $\tilde{\mathbf{x}}$, we have

$$\mathbf{x} - \tilde{\mathbf{x}} = \mathbf{x} - \boldsymbol{\mu} - \mathbf{U}\mathbf{z}$$

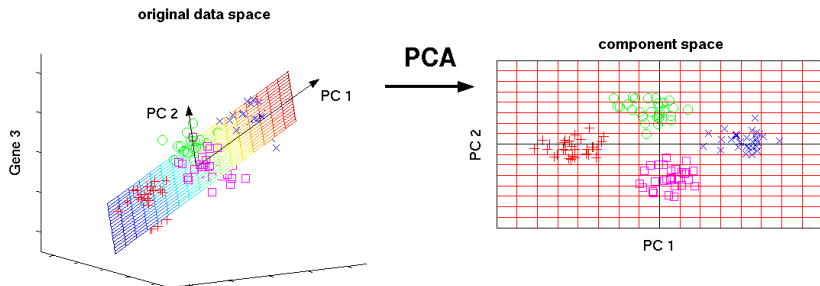
- Then, utilizing the definition of $\mathbf{z}$ and the orthonormality of $\mathbf{U}$, we have

$$\mathbf{U}^\top (\mathbf{x} - \tilde{\mathbf{x}}) = \mathbf{U}^\top (\mathbf{x} - \boldsymbol{\mu}) - \mathbf{U}^\top \mathbf{U} \mathbf{z} = \mathbf{z} - \mathbf{z} = \mathbf{0}.$$

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Dimensionality Reduction

- **Dimensionality reduction** aims to find a low-dimensional data vector to represent the original high-dimensional data vector.
- It can be implemented by **unsupervised learning** method or **supervised learning** method. In this lecture, we only introduce one typical unsupervised dimensionality reduction method, called **Principal Component Analysis (PCA)**.
- There are several usages of dimensionality reduction, such as
 - Visualization (as shown below)
 - Alleviate overfitting
 - Reduce the computational cost



Dimensionality Reduction

- The dimensionality of some types of data (*e.g.*, the image) is very high.
- As shown right, these colored images are from ImageNet, and the shape is $224 \times 224 \times 3$. Then, each image can be represented by a 150,528-dimensional vector.
- If the number of training data is not very large, then the learned model is likely to overfit, leading to poor performance on testing data.
- If we reduce the dimensionality before learning, then the overfitting could be alleviated, and the computational cost of learning will be reduced.



Dimensionality Reduction

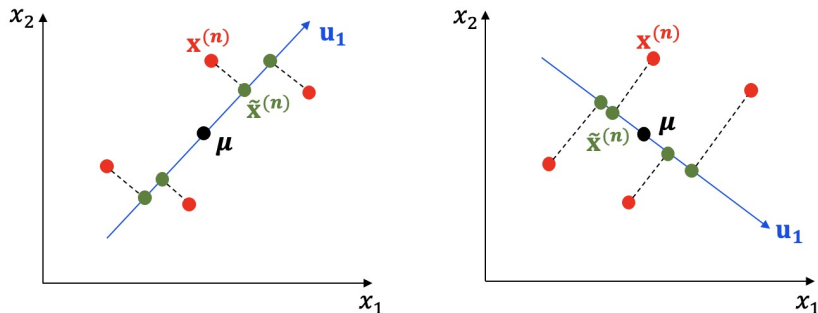
Dimensionality reduction:

- **Inputs:** given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \subset \mathbb{R}^D$, with D being the original dimension.
- **Goal:** find a K -dimensional ($K < D$) subspace $\mathcal{S}$, which consists of K orthonormal basis vectors $\{\mathbf{u}_k\}_{k=1}^K$, and $\mathbf{u}_i^\top \mathbf{u}_j = 0$ for $i \neq j$, while $\mathbf{u}_i^\top \mathbf{u}_i = 1, \forall i$. When projecting all points in $\mathcal{D}$ onto $\mathcal{S}$, it is desired that the structure or property of the original data is well preserved.
- **Outputs:** the basis vectors $\{\mathbf{u}_k\}_{k=1}^K$, and a new representation $\mathcal{D}' = \{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(N)}\} \subset \mathbb{R}^K$.

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Motivation



- In the above example, there is a 2-dimensional data set $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$, where $\mathbf{x}^{(n)} \in \mathbb{R}^2$.
- We aim to find a one-dimensional sub-space $\mathcal{S} = \{\mathbf{u}_1\} \in \mathbb{R}^2$, such that when projecting each point $\mathbf{x}^{(n)}$ onto this subspace, we obtain the corresponding reconstruction $\tilde{\mathbf{x}}^{(n)}$ and the representation z .
- According to your intuition, which subspace is better, **left** or **right**?

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Derivation 1: maximal variance

Principal Component Analysis:

- Given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \in \mathbb{R}^D$, we want to find a K -dimensional ($K < D$) subspace $\mathcal{S}$, which consists of K orthonormal basis vectors $\{\mathbf{u}_k\}_{k=1}^K$, such that the variance of the reconstructions $\tilde{\mathcal{D}} = \{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(N)}\}$ is maximal, i.e.,

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\tilde{\mathbf{x}}^{(n)} - \tilde{\boldsymbol{\mu}}\|^2, \quad (3)$$

where $\tilde{\boldsymbol{\mu}} = \frac{1}{N} \sum_{n=1}^N \tilde{\mathbf{x}}^{(n)}$ denotes the mean of the reconstructions.

- Utilizing the definitions of $\tilde{\mathbf{x}}^{(n)}$ (Eq. 1) and $\mathbf{z}^{(n)}$ (Eq. 2), it is easy to prove

$$\begin{aligned} \tilde{\boldsymbol{\mu}} &= \frac{1}{N} \sum_{n=1}^N \tilde{\mathbf{x}}^{(n)} = \boldsymbol{\mu} + \mathbf{U} \left(\frac{1}{N} \sum_{n=1}^N \mathbf{z}^{(n)} \right) \\ &= \boldsymbol{\mu} + \frac{1}{N} \mathbf{U} \mathbf{U}^\top \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu}) = \boldsymbol{\mu} \end{aligned} \quad (4)$$

- Substitute it into (3), we have

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\tilde{\mathbf{x}}^{(n)} - \boldsymbol{\mu}\|^2, \quad (5)$$

Derivation 1: maximal variance

Principal Component Analysis:

- Utilizing the definition $\tilde{\mathbf{x}} = \boldsymbol{\mu} + \mathbf{U}\mathbf{z}$, the above problem can be reformulated to

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{U}\mathbf{z}^{(n)}\|^2 \equiv \max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{z}^{(n)}\|^2. \quad (6)$$

- Substitute in the definition $\mathbf{z} = \mathbf{U}^\top (\mathbf{x} - \boldsymbol{\mu})$, we have

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})\|^2 \quad (7)$$

$$\equiv \max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \text{Trace}(\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \mathbf{U}). \quad (8)$$

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Derivation 2: minimal reconstruction error

Principal Component Analysis:

- Given a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \in \mathbb{R}^D$, we want to find a K -dimensional ($K < D$) subspace $\mathcal{S}$, which consists of K orthonormal basis vectors $\{\mathbf{u}_k\}_{k=1}^K$, such that the reconstruction loss between $\mathbf{x}$ and $\tilde{\mathbf{x}}$ is minimized, *i.e.*,

$$\min_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}^{(n)} - \tilde{\mathbf{x}}^{(n)}\|^2. \quad (9)$$

Two derivations are equivalent

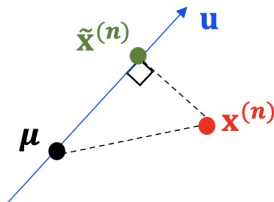
Theorem

Problem (5) and Problem (9) are equivalent, i.e.,

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\tilde{\mathbf{x}}^{(n)} - \boldsymbol{\mu}\|^2 \equiv \min_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}^{(n)} - \tilde{\mathbf{x}}^{(n)}\|^2. \quad (10)$$

Proof: By the Pythagorean Theorem, we have

$$\begin{aligned} & \underbrace{\frac{1}{N} \sum_{n=1}^N \|\tilde{\mathbf{x}}^{(n)} - \boldsymbol{\mu}\|^2}_{\text{projected variance}} + \underbrace{\frac{1}{N} \sum_{n=1}^N \|\mathbf{x}^{(n)} - \tilde{\mathbf{x}}^{(n)}\|^2}_{\text{reconstruction error}} \\ &= \underbrace{\frac{1}{N} \sum_{n=1}^N \|\mathbf{x}^{(n)} - \boldsymbol{\mu}\|^2}_{\text{constant}} \end{aligned}$$



- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

- Until now, we have known that PCA aims to solve the following optimization problem,

$$\max_{\mathbf{U}, \mathbf{U}^\top \mathbf{U} = \mathbf{I}} \frac{1}{N} \sum_{n=1}^N \text{Trace}(\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \mathbf{U}). \quad (11)$$

- We define the **empirical covariance matrix**, as follows:

$$\boldsymbol{\Sigma} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top. \quad (12)$$

- Then, the above optimization can be reformulated as follows:

$$\max_{\mathbf{U}} \text{Trace}(\mathbf{U}^\top \boldsymbol{\Sigma} \mathbf{U}) = \sum_{k=1}^K \mathbf{u}_k^\top \boldsymbol{\Sigma} \mathbf{u}_k, \text{ s.t. } \mathbf{U}^\top \mathbf{U} = \mathbf{I}. \quad (13)$$

PCA algorithm

- The Lagrangian function is formulated as follows:

$$L(\mathbf{U}, \mathbf{\Lambda}_K) = \text{Trace}(\mathbf{U}^\top \mathbf{\Sigma} \mathbf{U}) + \text{Trace}(\mathbf{\Lambda}_K^\top (\mathbf{I} - \mathbf{U}^\top \mathbf{U})), \quad (14)$$

where $\mathbf{\Lambda}_K = \text{diag}([\hat{\lambda}_1, \dots, \hat{\lambda}_K]) \in \mathbb{R}^{K \times K}$.

- Then, its optimal solution should satisfy

$$\frac{\partial L(\mathbf{U}, \mathbf{\Lambda}_K)}{\partial \mathbf{U}} = 2\mathbf{\Sigma} \mathbf{U} - 2\mathbf{U} \mathbf{\Lambda}_K = \mathbf{0} \quad (15)$$

$$\Rightarrow \mathbf{\Sigma} \mathbf{u}_k = \hat{\lambda}_k \mathbf{u}_k, \quad k = 1, \dots, K. \quad (16)$$

- It implies that the optimal primal solution $\mathbf{u}_k$ and the corresponding dual optimal solution $\hat{\lambda}_k$ are one of the **eigenvectors** and one of the **eigenvalues** of $\mathbf{\Sigma}$, which also satisfy the constraint $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$.

For matrix derivative, please refer to Wikipedia:

https://en.wikipedia.org/wiki/Matrix_calculus

PCA algorithm

- Utilizing SVD decomposition, we have

$$\Sigma = \mathbf{Q}\mathbf{\Lambda}_D\mathbf{Q}^\top = \sum_{i=1}^D \lambda_i \mathbf{q}_i \mathbf{q}_i^\top,$$

where $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_D] \in \mathbb{R}^{D \times D}$ with $\mathbf{q}_i$ being the eigenvector corresponding to the i -th largest eigenvalue λ_i , and $\mathbf{\Lambda}_D = \text{diag}([\lambda_1, \dots, \lambda_D])$ with $\lambda_1 \geq \dots \geq \lambda_D$.

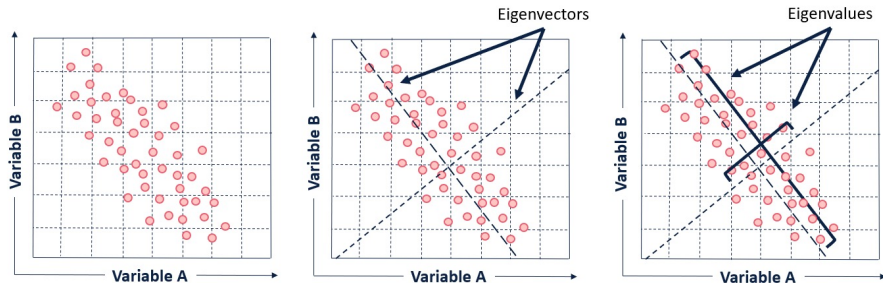
- Substitute into the objective function, we have

$$\sum_{k=1}^K \mathbf{u}_k^\top \Sigma \mathbf{u}_k = \sum_{k=1}^K \sum_{i=1}^D \lambda_i (\mathbf{u}_k^\top \mathbf{q}_i) \cdot (\mathbf{q}_i^\top \mathbf{u}_k) = \sum_{t \in T} \lambda_t,$$

where we utilize the property of eigenvectors (unit and orthogonal to each other), and $T \subset \{1, \dots, D\}$ with $|T| = K$ denotes the index subset of K picked eigenvalues.

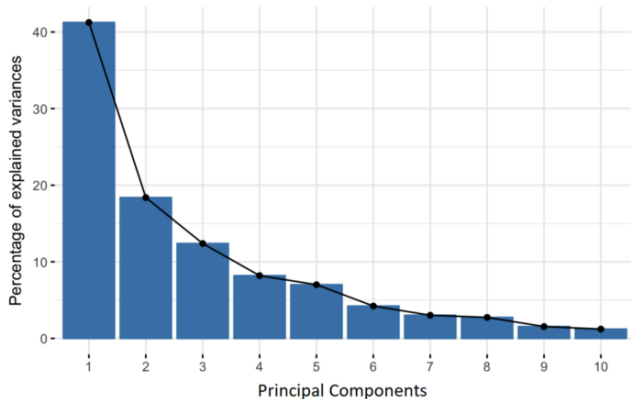
- It is obvious that we should pick the top- K eigenvalues. Correspondingly, the first K columns of $\mathbf{Q}$ should be used as the optimal solution to $\mathbf{U}$.

PCA algorithm



- One **eigenvector** corresponds to one of the basis vectors of the subspace obtained by PCA.
- One **eigenvalue** corresponds to the variance of the projected points on one basis vector (*i.e.*, eigenvector). A larger eigenvalue indicates more information about the original data.

PCA algorithm



- The variance/information of top- K eigenvectors (*i.e.*, top- K principal components) often takes a large percentage of the whole variance/information.
- Abandon the remaining components will not lose too much data information.

The above derivation of the optimal solution is summarized as the following steps:

- **Step 1:** Calculate the empirical covariance matrix $\Sigma = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top$
- **Step 2:** Do SVD decomposition of Σ to obtain its D eigenvalues $\{\lambda_i\}_{i=1}^D$ and eigenvectors $\{\mathbf{q}_i\}_{i=1}^D$, and rank them from large to small according to the eigenvalues.
- **Step 3:** Pick the top- K eigenvectors to form the matrix $\mathbf{U} = [\mathbf{q}_1, \dots, \mathbf{q}_K] \in \mathbb{R}^{D \times K}$
- **Step 4:** The new representation of $\mathbf{x}^{(n)}$ is $\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})$.

Examples

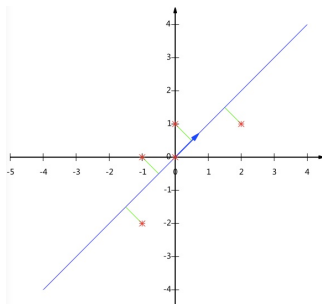
- Suppose that we have a set of 5 points in 2-dimensional space

$$X = \begin{pmatrix} -1 & -1 & 0 & 2 & 0 \\ -2 & 0 & 0 & 1 & 1 \end{pmatrix},$$

of which the mean column vector is $\boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

- We calculate its covariance matrix as

$$\boldsymbol{\Sigma} = \frac{1}{5} X X^T = \frac{1}{5} \begin{pmatrix} 6 & 4 \\ 4 & 6 \end{pmatrix}.$$



Examples

- SVD decomposition: we obtain

$$\mathbf{q}_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}, \mathbf{q}_2 = \begin{bmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}, \lambda_1 = 2, \lambda_2 = \frac{2}{5}.$$

- Thus, we set $\mathbf{U} = \mathbf{q}_1$
- The new representation is $\mathbf{U}^\top \mathbf{X} = \begin{pmatrix} \frac{-3}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 & \frac{3}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{pmatrix}.$

Reference:

Demo with code: <https://zhuanlan.zhihu.com/p/37777074>

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Decorrelation of the new representation

- After applying PCA, the original high dimensional data $\mathbf{x} \in \mathbb{R}^D$ to a low dimensional data $\mathbf{z} \in \mathbb{R}^K$
- **Interesting property**: the features of $\mathbf{z}$ are **decorrelated**.
- How to prove it? If the corresponding covariance matrix is diagonal, this means the features are uncorrelated.

Recall: Notation and Problem Setup

Given:

- Dataset $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\} \subset \mathbb{R}^D$
- Mean vector: $\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} \in \mathbb{R}^D$
- Covariance Matrix: $\boldsymbol{\Sigma} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \in \mathbb{R}^{D \times D}$

PCA Transformation:

- Let $\mathbf{U} = [\mathbf{q}_1, \dots, \mathbf{q}_K] \in \mathbb{R}^{D \times K}$ be the top- K eigenvectors of $\boldsymbol{\Sigma}$.
- Property: $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_K$ (Orthonormal columns).
- New representation (Code):

$$\mathbf{z}^{(n)} = \mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu}) \in \mathbb{R}^K$$

Goal: Prove that the dimensions of $\mathbf{z}$ are **uncorrelated**.

Derivation Step 1: Covariance of $\mathbf{z}$

The empirical covariance of the new representation $\mathbf{z}$ is defined as:

$$\text{Cov}(\mathbf{z}) = \frac{1}{N} \sum_{n=1}^N (\mathbf{z}^{(n)} - \boldsymbol{\mu}_z)(\mathbf{z}^{(n)} - \boldsymbol{\mu}_z)^\top$$

Note: Since $\mathbf{x}$ is centered ($\sum (\mathbf{x}^{(n)} - \boldsymbol{\mu}) = \mathbf{0}$), the mean of $\mathbf{z}$ is also zero:

$$\boldsymbol{\mu}_z = \mathbf{U}^\top \underbrace{\left(\frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu}) \right)}_{\mathbf{0}} = \mathbf{0}$$

Substitute $\mathbf{z}^{(n)} = \mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})$ into the covariance formula:

$$\begin{aligned} \text{Cov}(\mathbf{z}) &= \frac{1}{N} \sum_{n=1}^N \left[\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu}) \right] \left[\mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu}) \right]^\top \\ &= \frac{1}{N} \sum_{n=1}^N \mathbf{U}^\top (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \mathbf{U} \end{aligned}$$

Derivation Step 2: Matrix Simplification

Since $\mathbf{U}$ is constant with respect to the summation, we can pull it out:

$$\begin{aligned}\text{Cov}(\mathbf{z}) &= \mathbf{U}^\top \left[\frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})(\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \right] \mathbf{U} \\ &= \mathbf{U}^\top \boldsymbol{\Sigma} \mathbf{U}\end{aligned}$$

Recall Eigen-decomposition of $\boldsymbol{\Sigma}$: $\boldsymbol{\Sigma} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^\top$, where $\boldsymbol{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_D)$ and $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_D]$.

Since $\mathbf{U}$ consists of the first K columns of $\mathbf{Q}$ ($\mathbf{U} = [\mathbf{q}_1, \dots, \mathbf{q}_K]$):

$$\mathbf{U}^\top \mathbf{Q} = (\mathbf{I}_K, \quad \mathbf{0}_{K \times (D-K)})$$

Then, we have

$$\text{Cov}(\mathbf{z}) = (\mathbf{I}_K, \quad \mathbf{0}_{K \times (D-K)}) \boldsymbol{\Lambda} \begin{pmatrix} \mathbf{I}_K \\ \mathbf{0}_{(D-K) \times K} \end{pmatrix} = \text{top left } K \times K \text{ block of } \boldsymbol{\Lambda}$$

Thus, the covariance matrix of $\mathbf{z}$ is diagonal, implying that the features of $\mathbf{z}$ is **uncorrelated**.

Why is Decorrelation Useful? (1/2)

① Data Whitening (Preprocessing)

- We can scale each component by $1/\sqrt{\lambda_i}$ to get $\tilde{\mathbf{z}}$.
- $\text{Cov}(\tilde{\mathbf{z}}) = \mathbf{I}$ (Identity Matrix).
- Essential for algorithms assuming i.i.d. Gaussian inputs (e.g., certain layers in CNNs, ICA).

② Interpretability & Information Compression

- Each dimension z_k carries unique information (variance λ_k).
- No redundancy: We can drop dimensions with small λ_k without losing information contained in other dimensions.

Why is Decorrelation Useful? (2/2)

③ Handles Highly Correlated Features

- *Problem:* In Linear Regression, highly correlated features make $(X^T X)^{-1}$ unstable (ill-conditioned).
- *Solution:* PCA transforms features into orthogonal axes.
- *Result:* Principal Component Regression (PCR) yields stable coefficient estimates.

④ Accelerating Optimization

- *Problem:* Correlated features create elongated loss landscapes (narrow valleys), causing Gradient Descent to oscillate.
- *Solution:* Decorrelated inputs make the loss surface more spherical.
- *Result:* Faster convergence with larger learning rates.

- 1 Preliminary
- 2 Dimensionality Reduction
- 3 Derivations of Principal Component Analysis
 - Motivation
 - Derivation 1
 - Derivation 2
- 4 Principal Component Analysis Algorithm
- 5 Properties of Principal Component Analysis
- 6 Applications

Applying PCA to faces

- Consider running PCA on 2429 19×19 grayscale facial images (CBCL data), and each image is represented by a 361-dimensional column vector



- After running PCA, we can obtain several **eigenfaces**. With only top-3 eigenfaces, we achieve 79% accuracy on face/non-face discrimination on test data.
- We visualize the first 60 eigenvectors to the original shape, as follows:



Reference:

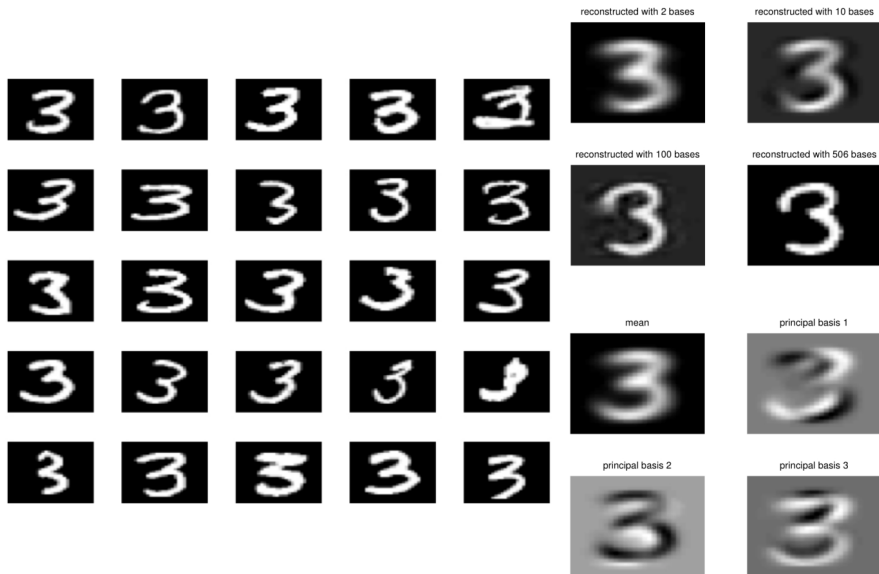
- Wikipedia: <https://en.wikipedia.org/wiki/Eigenface>
- Demo with code: https://scikit-learn.org/stable/auto_examples/applications/plot_face_recognition.html

Applying PCA to faces: Learned basis

Principal components of face images (“eigenfaces”)



Applying PCA to digits



Recap of PCA

- Dimensionality reduction aims to find a low-dimensional representation of the data.
- PCA projects the data onto a subspace which maximizes the projected variance, or equivalently, minimizes the reconstruction error.
- The optimal subspace is given by the top eigenvectors of the empirical covariance matrix.
- PCA produces a set of decorrelated features.

Further readings

Note that PCA is an orthogonal linear transformation method, thus it cannot handle non-linear data. There are some interesting variants of PCA, such as

- Kernel PCA: see Chapter 12.3 of Bishop's book ([Link](#))
- Probabilistic PCA: see Chapter 12.2 of Bishop's book
- Nonlinear PCA: see <http://www.nlpca.org/>
- Robust PCA: see https://en.wikipedia.org/wiki/Robust_principal_component_analysis

